

Web mô phỏng trực quan Thuật toán Dijkstra trên đồ thị

Nguyễn Duy Tuyên

MSSV: 2213821 - Lớp L02

Ngày 9 tháng 5 năm 2026

Nội dung trình bày

- 1 Giới thiệu
- 2 Cơ sở lý thuyết
- 3 Thiết kế & Hiện thực
- 4 Kết quả & Thảo luận
- 5 Kết luận

Giới thiệu đề tài

Web mô phỏng trực quan thuật toán Dijkstra trên đồ thị

Sinh viên: Nguyễn Duy Tuyên

MSSV: 2213821

Giảng viên: ThS. Hồ Thanh Phương



Hình: Giao diện Dashboard của ứng dụng

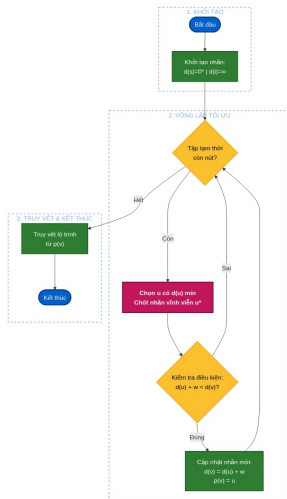
Lý do chọn đề tài

- Tối ưu hóa đường đi là bài toán cốt lõi trong nhiều lĩnh vực: logistics, bản đồ số, mạng máy tính và trí tuệ nhân tạo.
- Thuật toán Dijkstra (1959) là tiêu chuẩn vàng để tìm đường đi ngắn nhất trên đồ thị có trọng số không âm.
- Sinh viên thường gặp khó khăn khi tiếp cận qua công thức khô khan và bảng gán nhãn trừu tượng.
- **Giải pháp:** Xây dựng công cụ mô phỏng trực quan để biến khái niệm trừu tượng thành các bước minh họa sinh động.

- **Tính tương tác:** Tự do tạo, chỉnh sửa đồ thị (thêm/xóa nút, cạnh, trọng số) trực tiếp bằng chuột.
- **Tính trực quan:** Minh họa quá trình thuật toán "quét" qua các nút bằng hiệu ứng màu sắc và animation thời gian thực.
- **Tính giáo dục:** Cung cấp bảng nhật ký (log) chi tiết giải thích lý do chọn từng nút và cách cập nhật nhãn.

Bản chất thuật toán Dijkstra

- Phương pháp **tham lam (Greedy)**.
- Tìm đường đi ngắn nhất từ một nút nguồn đến tất cả các nút khác.
- Nguyên tắc **"loại trừ dần dần"**: liên tục chọn nút có khoảng cách tạm thời nhỏ nhất để chốt nhãn vĩnh viễn.
- **Điều kiện**: Trọng số cạnh phải **không âm**.



Hình: Lưu đồ thuật toán Dijkstra

Quy trình gán nhãn

Bộ nhãn: $L = [d, p]$

- d (Distance): Khoảng cách ngắn nhất tạm tính từ nguồn.
- p (Parent): Đỉnh cha ngay trước đó trên lộ trình.

Các giai đoạn chính:

- 1 **Khởi tạo:** $d(s) = 0$, các nút khác bằng ∞ .
- 2 **Cập nhật (Relaxation):** Kiểm tra nút kề v của nút u vừa chốt; nếu $d(u) + w(u, v) < d(v)$ thì cập nhật nhãn mới.
- 3 **Kết thúc:** Khi tất cả nút được chốt hoặc không còn đường đi khả thi.

Mã giả thuật toán (Pseudocode)

Logic thực thi trong Algorithm Engine

- **Khởi tạo:** $\text{dist}[\text{source}] = 0, \text{dist}[v] = \text{Infinity}$
- **Vòng lặp:** Trong khi tập unvisited còn nút:
 - 1 Chọn $u \in \text{unvisited}$ có $\text{dist}[u]$ nhỏ nhất.
 - 2 **Chốt nhãn vĩnh viễn** cho u .
 - 3 Với mỗi nút kề v của u , tính: $\text{alt} = \text{dist}[u] + \text{weight}(u, v)$
 - 4 **Nếu** $\text{alt} < \text{dist}[v]$: Cập nhật $\text{dist}[v] = \text{alt}$ và lưu vết $\text{prev}[v] = u$.

Phân tích độ phức tạp

Độ phức tạp thời gian:

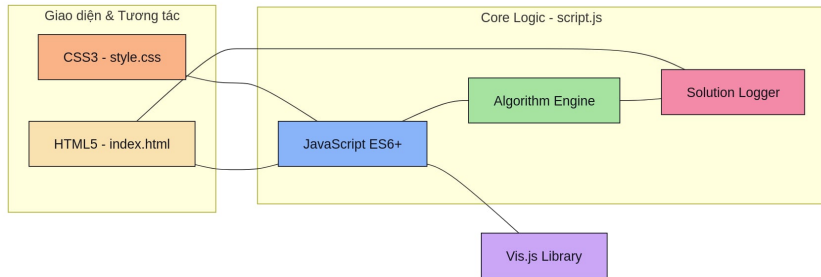
- Hiện thực bằng mảng/Set:
 $\mathcal{O}(|V|^2 + |E|)$.
- Phù hợp cho đồ thị quy mô vừa trên trình duyệt web.
- *Cải tiến*: Dùng Min-Heap để đạt $\mathcal{O}(|E| \log |V|)$.

Độ phức tạp không gian:

- Lưu trữ danh sách kề và mảng nhãn: $\mathcal{O}(|V| + |E|)$.
- Tối ưu cho bộ nhớ client-side.

- **HTML5 & CSS3 (Catppuccin Theme):** Thiết kế giao diện Dashboard hiện đại, giảm mỏi mắt, làm nổi bật các thành phần đồ thị.
- **JavaScript (ES6+):** Xử lý logic thuật toán trực tiếp tại trình duyệt, đảm bảo phản hồi tức thời.
- **Vis.js Network:** Thư viện chuyên dụng hỗ trợ vẽ đồ thị động, xử lý kéo thả và tương tác vật lý giữa các nút.

Kiến trúc hệ thống



Hình: Mô hình kiến trúc phân lớp của simulator

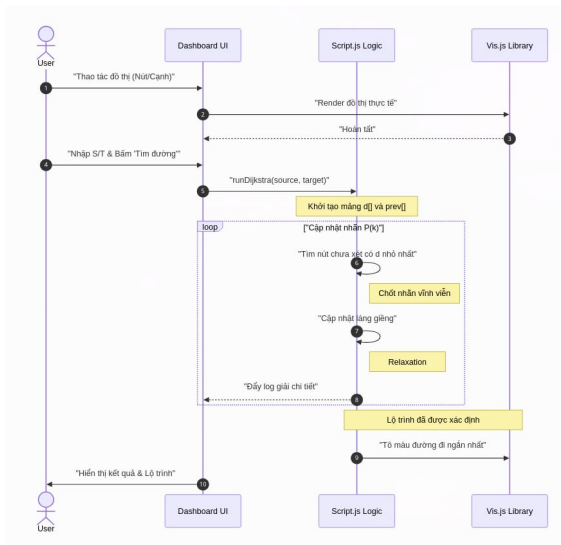
Các thành phần cốt lõi:

- **Algorithm Engine:** Logic Dijkstra.
- **Visualizer:** Hiệu ứng & Animation.
- **UI Layer:** Tương tác người dùng.

Cấu trúc dữ liệu:

- **Adjacency List:** Lưu trữ đồ thị.
- **DataSet (Vis.js):** Quản lý nút/cạnh động.
- **HTML Log:** Hiển thị bước giải.

Trình tự xử lý dữ liệu



Hình: Sequence Diagram từ lúc bắt đầu đến khi có kết quả

Giao diện Dashboard trực quan



Hình: Toàn cảnh môi trường mô phỏng

Khả năng truy vết và Đối chiếu

Tính hiệu quả giáo dục:

- Highlight lộ trình ngắn nhất bằng màu hồng nổi bật.
- Log chi tiết giải thích logic chọn nút.
- Công thức đối chiếu: $Nhãn(j) + d(j,n) = Nhãn(n)$.

BƯỚC 2: XÉT NÚT 12

- Chọn nút **12** có nhãn tạm thời nhỏ nhất (1).
- Chốt nhãn vĩnh viễn cho nút 12.
- + Cập nhật nút 11: $\infty \rightarrow 5$

$$P(2) = \{ 0^*, 7, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, 5, 1^* \}$$

TRUY VẾT LỘ TRÌNH

- Bắt đầu từ nút đích **12** lùi về nút nguồn **1**.
- **Quy tắc:** Tìm nút j đã chốt nhãn sao cho: $Nhãn(j) + d(j,n) = Nhãn(n)$.

→ Tại nút **12** ($L=1$): Nút đứng trước là **1** vì:
 $Nhãn(1) + d(1,12) = 0 + 1 = 1$ (Khớp!).

Hình: Chi tiết quá trình truy vết lộ trình

Xử lý các trường hợp ngoại lệ

- **Đồ thị không liên thông:** Hệ thống nhận diện khi không còn nút khả thi để xét và thông báo: *"Không tìm thấy đường đi"*.
- **Trọng số âm:** Ngăn chặn ngay từ lớp giao diện (Input validation) để đảm bảo tính đúng đắn của thuật toán Dijkstra.
- **Đồ thị có hướng/vô hướng:** Cho phép chuyển đổi linh hoạt chế độ đồ thị thông qua Checkbox.

Kết luận và Hướng phát triển

- **Kết quả:** Hoàn thành ứng dụng mô phỏng đáp ứng đầy đủ tính chính xác và tính trực quan giáo dục.
- **Giá trị:** Giúp sinh viên nắm bắt bản chất thuật toán nhanh chóng hơn so với cách học truyền thống.

Hướng phát triển tương lai:

- Hỗ trợ thêm: Bellman-Ford, A*, Floyd-Warshall.
- Tối ưu hiệu suất cho đồ thị cực lớn (Big Data).
- Xây dựng nền tảng học thuật toán đồ thị trực tuyến.

Tài liệu tham khảo

- ❶ E. W. Dijkstra, "A note on two problems in connexion with graphs," 1959.
- ❷ T. H. Cormen et al., *Introduction to Algorithms*, MIT Press, 2009.
- ❸ Phan Quốc Khánh, *Quy hoạch tuyến tính và các bài toán tối ưu trên đồ thị*, 2002.
- ❹ Vis.js Community, "Vis-network Documentation," 2026.

TRUY CẬP MÃ NGUỒN VÀ BẢN DEMO



Bản chạy thử (Web)



Mã nguồn (GitHub)

CẢM ƠN CÔ VÀ CÁC BẠN

ĐÃ LẮNG NGHE BÀI THUYẾT TRÌNH

Em xin sẵn sàng nhận các câu hỏi và ý kiến đóng góp.